Predicting NBA Shot Make Probability from Tracking Data

Daniel Acevedo Vega

1 Summary

The final model is a **gradient-boosted tree model (XGBoost)**, benchmarked against a logistic regression baseline (a blend of the two was tested; the optimal logistic weight was zero). Its inputs are shot geometry, defender tracking, shooter context, and an **out-of-fold, empirical-Bayes-shrunk estimate of each shooter's skill** by shot family. The test file is a season (d7a2d) that is not in either training season, so every number below comes from **leave-one-season-out validation**: train on one season, score the other.

	Held-out log-loss
Always predict league make rate	0.6896
Logistic baseline	0.6387
Final model	0.6245

2 Data and validation design

- **425,719 training shots** from two seasons (fe055, 2676a) and 213,977 test shots from a third. Because the test season is new, random k-fold CV would be slightly optimistic: the same games, lineups, and season-level shooter form would show up in both train and validation. I hold out a whole season instead, and report the pooled log-loss over both held-out seasons.
- About 99% of test shots come from shooters who also appear in training, so historical shooter skill carries over.
- Data checks that shaped features:
  - `distance` equals the Euclidean distance from $(x, y) = (-41.75, 0)$ to within 10^{-10}, which fixes the hoop and lets me derive angle and corner-three geometry.
  - `shotclock` has no NAs, but values of exactly 0 and exactly 24 appear ~10k times each, which suggests codes for "clock off" or "just reset". I flag both.
  - `closestdefapproach` is a text array `{d_1.0s, d_0.75s, d_0.5s, d_0.25s}`, which I parse into numbers.

3 Features

Group	Features
Location	distance, depth from hoop, lateral offset, shot angle, three, corner-three flag, heave flag
Shot type / state	one-hot shot type, game state (halfcourt / transition / second chance), month

Group	Features
Defense	closest-defender distance at release, # contesters, mean contester distance, contest pressure $\sum 1/d$, defender distance 0.5s before, **closing speed** = distance 1s (and 0.25s) before release minus distance at release
Shooter context	shooter speed, dribbles before, shot clock, shot-clock = 0 / = 24 flags, late-clock ($\leq$ 4s) flag
Player skill	shooter makes-above-expected for rim / midrange / three (plus the one matching this shot), closest-defender makes-allowed-above-expected. (Shooter volume and 3PA rate were tried and dropped; see Section 7.)

3.1 Shooter skill without leakage

"Expected" is an **out-of-fold context-only XGBoost**, which uses every feature except player identity. For shooter i in shot family f:

$$\text{skill}_{if} = \frac{\sum(y - \hat{p})}{n_{if} + k_f}, \qquad k_f = \frac{\bar{\sigma}_f^2}{\hat{\tau}_f^2}$$

$\hat{\tau}_f^2$ is the true between-shooter variance, estimated by method of moments (observed variance of per-shooter mean residuals minus the expected binomial noise). A shooter with few attempts is pulled toward 0 (league average given the shot), and a high-volume shooter keeps most of their residual. Estimated k (the "prior attempts"): three = 173, mid = 165, rim = 152. Three-point skill needs the most attempts before it is trusted, consistent with how noisy three-point percentage is.

Leakage control is two-level. Both $\hat{p}$ and the skill value for a training row are computed from **other folds only**. Rows being scored (the held-out season, or the test set) get skill computed from all training rows, which is exactly the situation the real test shots are in.

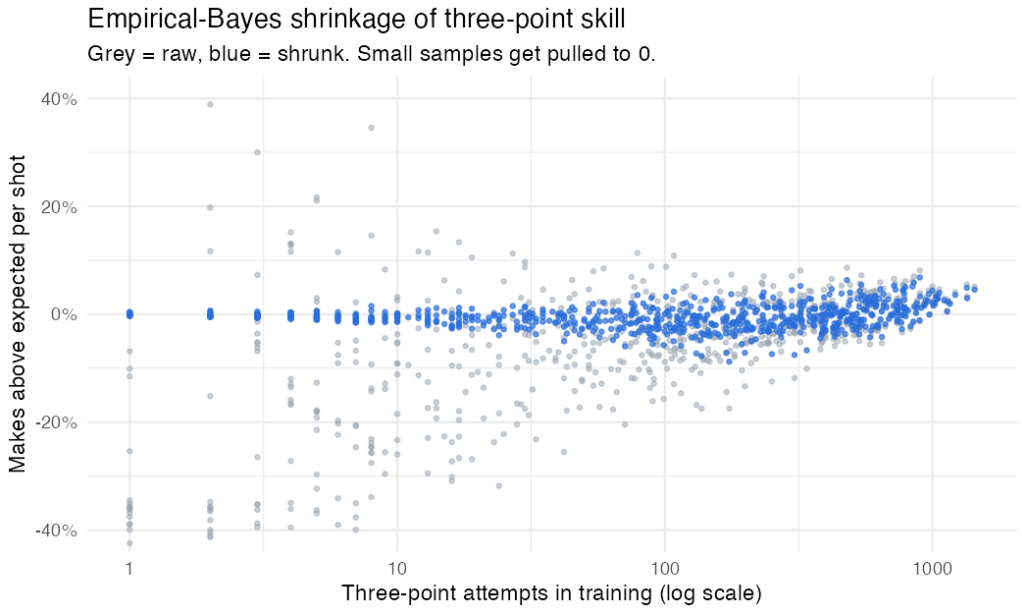


Figure 1: Raw vs. shrunk three-point skill. Low-volume shooters' extreme raw values collapse toward 0.

4 Model progression

Table 3: Held-out log-loss (lower is better). Rows marked x are diagnostic experiments, not candidates.

Model	Pooled	Hold out fe055	Hold out 2676a
1. Logistic baseline (distance, shot type, three, defender dist.)	0.6387	0.6370	0.6405
2. XGBoost, context features only	0.6264	0.6247	0.6280
x. XGBoost + skill computed in-sample (leaky)	0.6263	0.6244	0.6283
3. XGBoost + raw (unshrunk) out-of-fold skill	0.6251	0.6236	0.6265
x. XGBoost + shrunk skill + shooter volume/3PA rate	0.6266	0.6259	0.6273
4. XGBoost + shrunk out-of-fold skill	0.6245	0.6229	0.6261
5. Blend (1.00 XGBoost / 0.00 logistic)	0.6245	0.6229	0.6261
6a. Blend + Platt scaling (cross-fitted)	0.6246	0.6230	0.6262
6b. Blend + isotonic (cross-fitted)	0.6250	0.6236	0.6263

Reading the table top to bottom:

1. **Logistic baseline, 0.6387.** Distance splines, shot type, three, and closest-defender distance already capture most of the easy signal (the null model scores 0.6896).
2. **Context XGBoost, 0.6264** (0.0124 better). This is the biggest single step. The gain comes from interactions a linear model can't express. For example, the raw make rate is *not* monotonic in defender distance, because wide-open shots are disproportionately long threes, while at a fixed location more space always helps. Trees learn that conditional effect.

3. **+ raw out-of-fold skill, 0.6251.** Real signal, but noisy for low-volume shooters.
4. **+ shrunk skill, 0.6245** (a further 0.0006). Shrinkage turns shooter identity into a stable feature. It improves both held-out seasons.
5. **Blend.** The log-loss-optimal weight on the logistic model was 0.00: the boosted model already contains everything the baseline knows. I kept the logistic model as an interpretable anchor, not as a production component.
6. **Calibration.** Platt and isotonic, each fit on one held-out season and applied to the other, did *not* improve log-loss (the calibration plot below shows why), so the final predictions are the uncalibrated XGBoost output.

The two rows marked **x** are experiments that went into the "what didn't work" section.

5 What drives the prediction

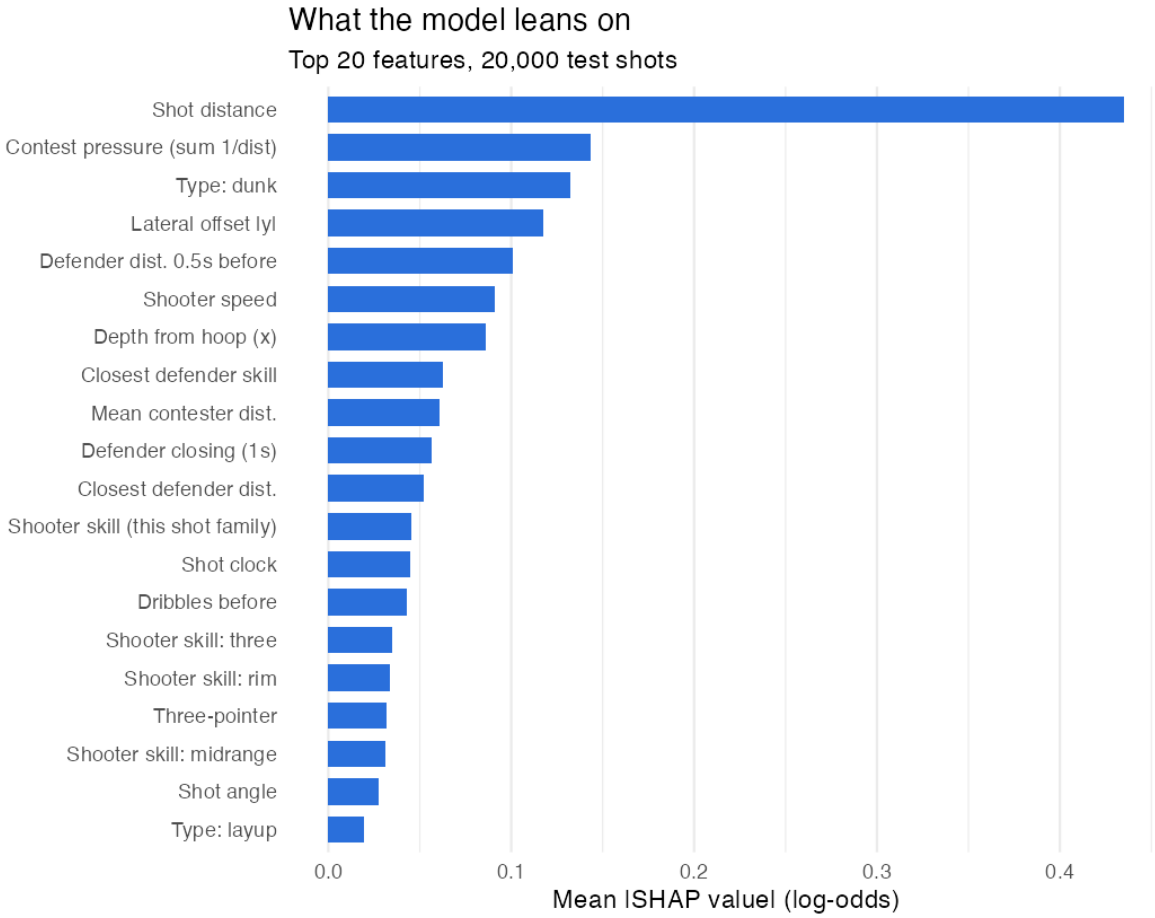


Figure 2: Mean absolute SHAP value (contribution to log-odds) on 20,000 test shots.

Table 4: SHAP importance summed by feature group.

Feature group	Share of total |SHAP|
Location	40.1%

Feature group	Share of total |SHAP|
Defense (tracking)	25.7%
Shooter context / game state	11.4%
Shot type	10.8%
Shooter skill	8.4%
Defender skill	3.6%

- **Location dominates.** Shot distance alone is the largest single feature. Every foot matters most near the rim, where the curve is steepest (SHAP dependence, bottom right).
- **Defense is the second pillar**, but it is spread across several correlated tracking features: contest pressure, the defender's position 0.5s before release, mean contester distance, closing speed, and release distance. Summed as a group it is second only to location.
- **Shooter speed** hurts: shots taken while moving fast (off-balance, on the run) go in less often at the same spot.
- **The shot clock** mostly matters at the very end: shots with under ~4 seconds left are clearly worse (forced shots).
- **Shooter skill** is modest per shot on average, but it is what separates a great shooter from a poor one at the same spot. Its SHAP dependence is close to linear and monotone, which is the behavior you want from a skill estimate.

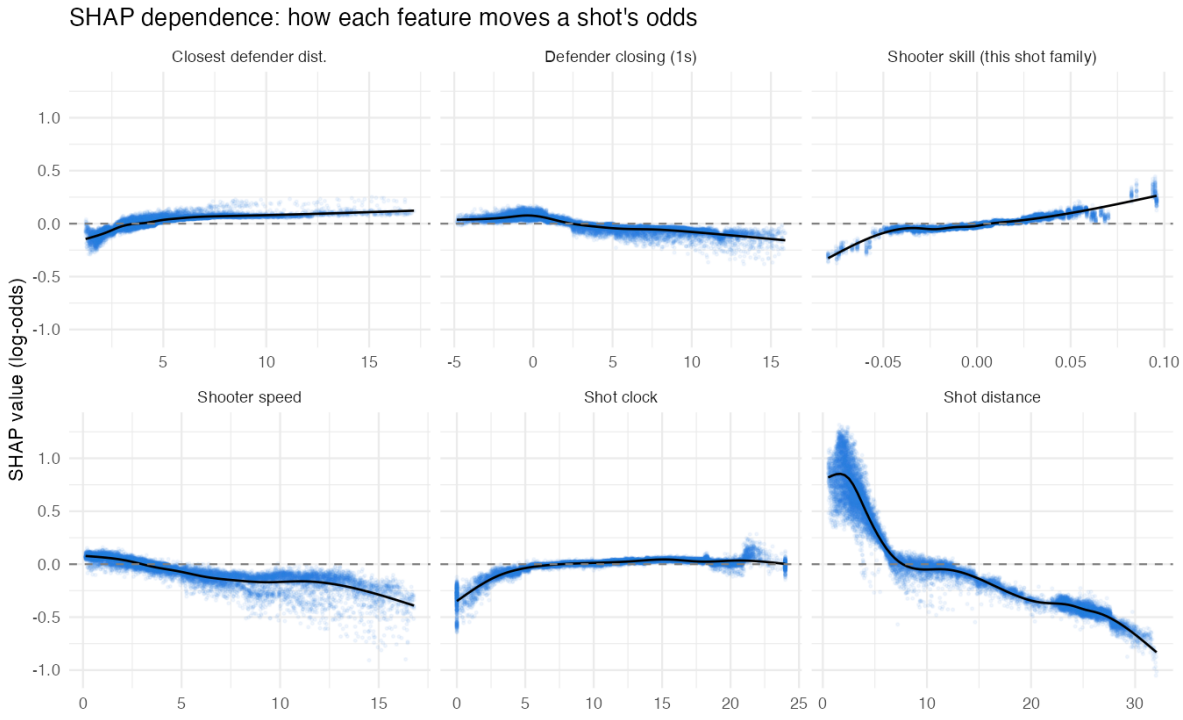


Figure 3: SHAP dependence: each point is a test shot, the line is a smoother.

5.1 Partial dependence: the basketball logic on jumpers

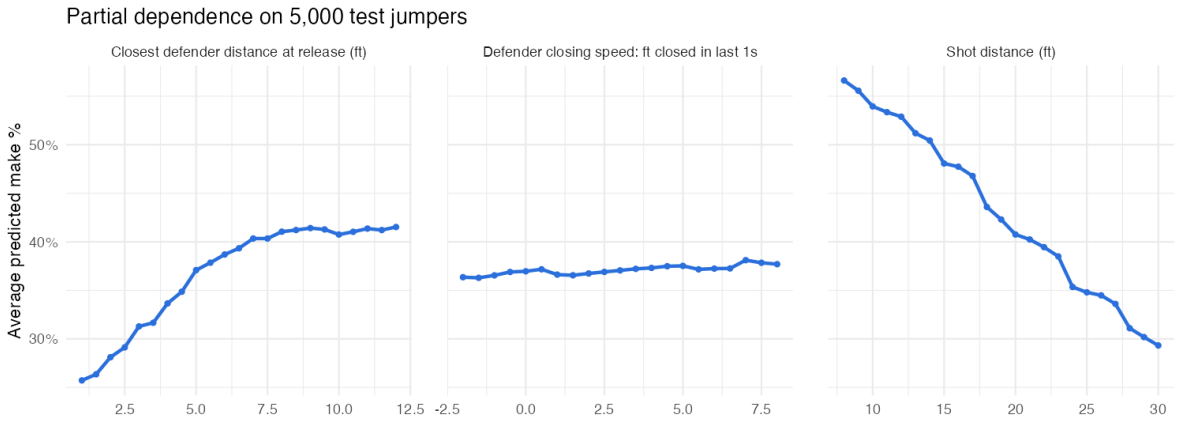


Figure 4: Average predicted make probability over 5,000 test jumpers as one input is swept. For distance, each shot slides along its own ray from the hoop, with three-point status and location features recomputed.

Partial dependence plots are only honest if the swept input moves the way it does on a real court. The tracking features are highly correlated, so I move them together. Distance slides each shot along its own ray, recomputing three-point status and location. Defender distance moves the whole defender, including their position 0.5s earlier and their contribution to contest pressure. Closing speed sets a linear approach path that ends at the real release distance. (A first version swept one column at a time. It produced nearly flat defender curves, because the model credited the effect to the correlated columns that were held fixed.)

On an average test jumper:

- **Distance:** 54.0% at 10 ft, 40.8% at 20 ft, 34.5% at 26 ft. There is a visible step at the three-point line, where shot selection and mechanics change.
- **Space:** a jumper with the closest defender at 2 ft goes in 28.1% of the time, versus 38.7% at 6 ft, a gain of about **10.6 percentage points** for four extra feet.
- **Closing speed:** I expected "a closing defender costs X%". The data says something subtler. *Holding the release distance fixed*, a defender who closed 5 ft in the last second changes the make probability by only **+0.6 percentage points** compared with a stationary one. That makes sense on a court: if the defender ends at the same spot, a fast closeout means the defender was *farther away* while the shooter gathered, so the shooter had more room during the motion. The contest that hurts is the one that is *there at release*. Closing speed matters mostly through where the defender ends up, which the release-distance curve already captures. The SHAP dependence for closing speed is negative only because it is conditioned on the defender's position 0.5s earlier, which is a different question.

6 Calibration

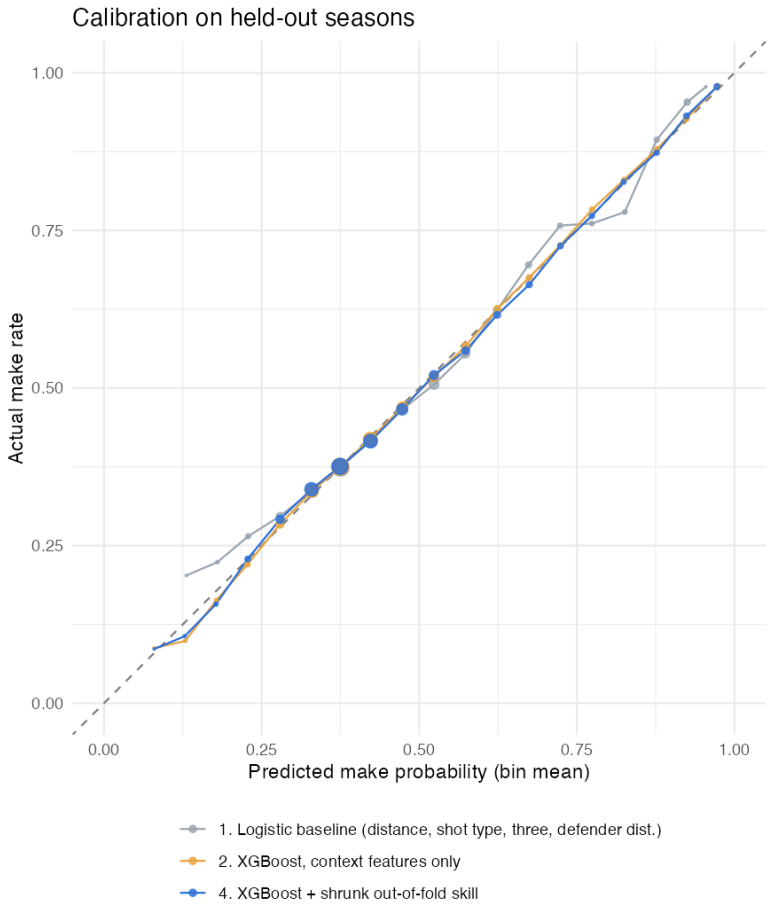


Figure 5: Predicted vs. actual make rate in 5-point bins on the held-out seasons.

All three models track the diagonal closely on seasons they never saw. The logistic baseline is the least calibrated at the extremes (it under-predicts shots it rates at 10–25%, where its linear distance terms can't bend enough for long pull-ups and heaves). Both XGBoost models are within a point or two in every bin with meaningful volume. That is why cross-fitted Platt and isotonic calibration didn't reduce log-loss: there is no systematic bias left to remove, and isotonic adds step-function noise.

7 What didn't work, and pitfalls

- **My first skill model made predictions *worse*.** With shooter volume and 3PA-rate features included, the skill model scored 0.6266, worse than using no skill at all (0.6264). The clue was early stopping: it stopped after 195 / 334 rounds (one per held-out season) versus 522 / 634 for the final skill model, a sign of overfitting to something that doesn't transfer. A separate diagnostic (not part of the final script) showed the skill signal itself was real: adding cross-season shrunk skill to a frozen context prediction in a logistic regression improved log-loss on the other season. The culprit was the **volume features**. Inside training they are counted out-of-fold (on 4/5 of the rows), but for the scored season on all rows, so their scale shifts between fit time and prediction time. Dropping them gave 0.6245. The lesson: out-of-fold features must also have the

same distribution at fit and predict time, not just no leakage.

- **Raw FG% vs. shrinkage.** Raw out-of-fold makes-above-expected helps less than the shrunk version (0.6251 vs 0.6245). A shooter who went 3-for-3 from deep looks like the best shooter in the league. Empirical Bayes needs ~173 three-point attempts before it trusts half of a shooter's residual.
- **Leakage is real, just smaller than I expected.** Computing skill *in-sample* (each shot's own outcome included) scored 0.6263, no better than having no skill feature, and worse than doing it properly. The effect is modest here because each shot is weighted by only $1/(n+k)$ with $k \approx 150+$. With smaller k or target-encoded IDs it would be much worse.
- **One-at-a-time partial dependence lied** about defense (see above). Correlated tracking features need coherent perturbations.
- **Random k-fold would have been optimistic.** The test file is an unseen season, so I validated by season. One small optimism remains that I want to flag: the number of boosting rounds was chosen by early stopping on the held-out season. The final model uses 665 rounds (1.15x the mean best iteration, since it trains on twice the data).
- **Shot clock at exactly 0 or 24** behaves like a code, not a measurement (e.g. clock off at the end of a period). I flagged it rather than treating it as real time remaining.

8 What I would add with more time or data

- **Shooter fatigue / load**: minutes played in the game, time since last rest, back-to-backs. Month is a weak proxy.
- **Catch-and-shoot vs. pull-up, and pass origin**: `dribblesbefore` $= 0$ approximates catch-and-shoot but not the pass type (skip pass, drive-and-kick) or the time from catch to release.
- **Body mechanics**: shooter body orientation relative to the rim, feet set vs. moving laterally, release height and release time, and defender hand/closeout direction (a hand in the face matters more than distance).
- **Defender identity at scale**: a proper defender-contest model, e.g. a hierarchical model with shooter and defender random effects fit jointly rather than two separate residual passes.
- **Hierarchical skill across families**: borrow strength between a shooter's midrange and three-point skill (a player good at one tends to be good at the other) with a multivariate prior.
- **Time-aware skill**: weight recent shots more, since skill drifts across seasons. The leave-one-season-out results show how much signal survives a season boundary.
- **Tuning**: a small Bayesian hyperparameter search and a monotone constraint on closest-defender distance within shot family.

9 Reproducibility

`Rscript project_code.R` (R 4.6.1, xgboost, data.table, ggplot2, splines, R.utils) runs everything and fills `submission.csv`. Seed fixed at 2027. This write-up (`project_writeup.Rmd`) only reads the saved outputs in `results/` and `figures/`.